Lecture 9 — Detailed English Notes

Based on the uploaded Hindi lecture transcript

Lecture focus: Git and GitHub version control, MongoDB Atlas cloud database, environment variables and secrets, and deploying an Express/Node.js API to Render.

Important transcript note: The lecture demonstrates a hotel/restaurant API project with routes such as people/persons and menu items. A few spoken values and command names were unclear in the auto-transcription, so the notes preserve the intended workflow and flag places where you should use the exact values from your own project.

1. Why deployment and security matter

Writing an API that runs on your own computer is only one part of building an application. Other people need a way to access it, which means the application and its database must be hosted somewhere reachable online.

- Version control helps track code changes and recover a previously working version.
- A remote repository provides an online backup and makes collaboration easier.
- A cloud database lets the deployed API access data even when your personal laptop is off.
- Secrets such as database usernames, passwords, and connection strings must not be committed to a public repository.
- Deployment runs the Node.js server on a hosting provider so users can reach it through a public URL.

2. Git: version control

Git is a version-control system. It records project history as commits (snapshots of tracked files). If a later code change breaks the application, you can inspect earlier commits and restore a working state instead of manually undoing changes across many files.

2.1 Install and verify Git

```
git --version
```

Download Git from the official Git website if it is not installed. Run the command above in a terminal; a version number confirms that Git is available.

2.2 Initialize a repository

Open a terminal in the project's root folder and initialize Git once for that project:

```
git init
git status
```

git status reports untracked, modified, staged, and committed changes. VS Code may also show file colors or decorations, but those visual indicators are provided by the editor; the Git commands are what manage the repository.

2.3 Stage files and create a commit

Staging tells Git which changes should be included in the next snapshot. You can stage one file or all eligible changes.

```
git add server.js
git add .
git status
git commit -m "Initial working version
```

- git add server.js stages only that file.
- git add . stages eligible changes under the current directory.
- git commit -m "message" creates a named snapshot from staged changes.
- A useful commit message says what changed, such as “Add menu item routes” or “Read port from environment”.

A commit is local until you push it to a remote repository.

3. .gitignore and node_modules

The node_modules directory contains installed dependencies. It is generally not committed because it can be recreated from package.json and the lockfile. Git should track the project's dependency declarations, not the installed dependency folder.

3.1 Create a .gitignore file

```
.env
node_modules/
npm-debug.log*
.DS_Store
```

Add other temporary files or local-only files when appropriate. The exact ignore list depends on your project. A .gitignore rule prevents untracked matching files from being added in future; it does not automatically remove a file that Git is already tracking.

3.2 If node_modules was already tracked

```
git rm -r --cached node_modules
git add .gitignore
git commit -m "Stop tracking node_modules
```

The --cached option removes the directory from Git's index while leaving your local files in place. If a sensitive file such as .env was already committed, simply adding it to .gitignore is not enough; remove it from tracking and rotate any exposed credentials.

4. GitHub: remote repository

GitHub hosts Git repositories online. The lecture creates a repository for the local project, connects the local repository to it, pushes commits, and later pushes new commits as the project changes.

4.1 Connect a local project to GitHub

Create a repository on GitHub, then use the remote URL shown by GitHub. Follow the commands GitHub displays for your repository; a typical HTTPS setup looks like this:

```
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git branch -M main
git push -u origin main
```

Replace USERNAME and REPOSITORY with your actual values. Authentication may use a browser login, credential manager, or SSH key depending on your setup. Do not paste passwords or tokens into public code or chat.

4.2 Normal update workflow

```
git status
git add .
git commit -m "Describe the change"
git push
```

The lecture's key idea is to commit meaningful working checkpoints and push them so the remote repository stays updated. If a collaborator changes the remote repository, use git pull to bring those changes into your local branch. Pulling can require resolving conflicts if both sides changed the same parts of a file.

4.3 Git terms to remember

- **Working tree:** your current files.
- **Staging area:** the changes selected for the next commit.
- **Commit:** a snapshot in the local history.
- **Remote:** a linked repository hosted elsewhere, such as GitHub.
- **Push:** send local commits to the remote.
- **Pull:** fetch remote changes and integrate them into the current branch.
- **Branch:** a separate line of development; the lecture uses main.

5. MongoDB Atlas: host the database online

The lecture moves the database from a local MongoDB instance to MongoDB Atlas, MongoDB's managed cloud database service. With a cloud database, the API can read and write data online without relying on the developer's laptop being switched on.

5.1 High-level setup

- Create/sign in to a MongoDB Atlas account.
- Create a project and a free-tier cluster if available for your account and region.
- Create a database user with a strong, unique password.
- Configure network access so the deployed service is allowed to connect. Prefer restricting access to the necessary IP addresses or supported network options rather than leaving broad access open.
- Choose the application connection method and copy the connection string.
- Replace the placeholder password in the connection string carefully, and use the actual database name and cluster details for your project.

Atlas screens, free-tier availability, and setup options can change. Follow the current dashboard instructions rather than relying on a particular button label from the recording.

5.2 Local database versus cloud database

```
Local example:
mongodb://127.0.0.1:27017/hotel

Atlas example:
mongodb+srv://<db_user>:<db_password>@<cluster-host>/<database>?retryWrites=true&w=majority
```

These are illustrative patterns, not your real credentials. Keep the complete Atlas URI private. If you change the connection string in the application, subsequent database operations use the database identified by that string. Existing local records do not automatically appear in Atlas; data must be migrated or inserted separately.

5.3 Test the connection

Start the API and use Postman to test an endpoint, for example GET /person or GET /menu. Create a record through the relevant POST route, then inspect the Atlas collections to verify that the document was stored in the cloud database.

- If a GET endpoint returns an empty list, check whether the cloud database has records; your local database's records may not have been copied.
- If the connection fails, check the URI, username/password, network access rules, and the server logs.
- Do not assume that a successful API response proves that the record was written to the database you intended; verify the target database.

6. Environment variables and the .env file

A database connection string can contain a username and password. Hard-coding it in source code is unsafe, especially if the repository is public. The lecture introduces environment variables to separate configuration and sensitive values from application code.

6.1 Install dotenv

```
npm install dotenv
```

This adds dotenv as a project dependency and updates the package metadata/lockfile.

6.2 Create .env in the project root

```
PORT=3000
MONGO_URI=mongodb+srv://<db_user>:<db_password>@<cluster-host>/<database>?retryWrites=true&w=majority
```

Use the variable names your application expects. Do not include spaces around the equals sign, and do not wrap values in extra quotation marks unless required by your configuration. Never commit the real .env file.

6.3 Load variables in the application

```
require("dotenv").config();

const PORT = process.env.PORT || 3000;

// Example database configuration:
// use process.env.MONGO_URI in your existing db connection code
```

The lecture shows the same core idea: load dotenv near the beginning of the application and read values through process.env. Keep your project's existing MongoDB connection logic; update only the connection-string source rather than replacing the whole database module blindly.

6.4 Ignore .env

```
.env
node_modules/
```

If the project needs to tell collaborators which variables are required, create a .env.example containing placeholder values only, for example PORT=3000 and MONGO_URI=replace_me. Share actual secrets securely, not through a public repository.

6.5 Set environment variables on the hosting platform

The deployed service does not automatically receive the .env file from your laptop if that file is ignored by Git. Add the required variables in the host's service settings. In the lecture, the PORT and MongoDB URI values are entered in the deployment environment-variable section.

Security warning: The transcript appears to show a simple database password during the demonstration. Treat any credential that was exposed in a recording, repository, screenshot, or shared transcript as compromised. Change/rotate it before using the database for anything important.

7. Configure the port correctly

A hosting provider may assign a port through an environment variable. The server should listen on that port when supplied, while using a local fallback during development.

```
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});
```

The exact code depends on how your server is structured. Do not hard-code a port that conflicts with the hosting provider's runtime configuration. The lecture's intended pattern is process.env.PORT with 3000 as the local fallback.

8. Deploy the Node.js API to Render

Render is used in the lecture to deploy the Node.js/Express web service connected to the GitHub repository. Once deployed, Render provides a public URL that can be used instead of localhost.

8.1 Pre-deployment checklist

- The project is pushed to GitHub and the correct branch (shown in the lecture as main) is selected.
- package.json contains the required dependencies and a valid start script.
- The app listens on process.env.PORT || 3000.
- The database connection reads the URI from process.env.MONGO_URI (or your chosen variable name).
- The .env file and node_modules directory are not committed.

- The Atlas database user and network access settings allow the deployed service to connect.

8.2 Create the web service

- Sign in to Render and create a new Web Service.
- Connect the GitHub repository that contains the API.
- Choose the branch to deploy, usually main in this lecture.
- Use the Node runtime if asked; leave the root directory blank if the Node project is at the repository root.
- Set the build command to install dependencies, typically npm install.
- Set the start command to the script your package.json defines, commonly npm start. Do not confuse it with npm run dev unless the project is specifically configured for that.
- Choose a plan available to your account; free-plan availability and behavior may change.
- Add environment variables such as PORT if required by the platform, and the MongoDB connection string. Use the exact variable names referenced by your code.
- Create/deploy the service and inspect the build and runtime logs.

8.3 Test the public URL

After a successful deployment, copy the service URL provided by Render. Replace localhost:3000 in your Postman request with the deployed host, then append the same route path.

```
Local:
http://localhost:3000/person

Deployed (illustrative):
https://your-service-name.onrender.com/person
```

Use the actual URL Render gives your service. Test GET and POST routes with Postman. If the API responds and new records appear in MongoDB Atlas, the API and database are communicating online.

8.4 Common deployment failure: wrong start command

The transcript describes an initial deployment failure caused by a command mismatch, then correcting the start command. Check package.json. If it contains a script such as "start": "node server.js", use npm start as the Render start command. If the entry file or script name differs, use the one actually defined by your project.

8.5 Troubleshooting

- **Build fails:** read the first meaningful error in the build logs; check package.json, dependency names, and the build command.
- **Service starts then exits:** check the start command, entry file, and runtime logs.
- **Database connection error:** verify the environment variable name/value, Atlas database user, password, and network access.
- **API works locally but not online:** check process.env.PORT, deployment logs, and whether all required environment variables are configured on Render.
- **Endpoint returns no data:** confirm the route path and whether the selected Atlas database/collection contains documents.
- **Changes do not appear online:** commit and push the latest code, then confirm that Render deployed the intended branch/commit.

9. Full workflow: from local code to public API

- Build and test the Express API locally.
- Initialize Git and commit a working version.

- Create a GitHub repository and push the project.
- Add .gitignore rules for .env, node_modules, and local-only files.
- Create a MongoDB Atlas cluster and database user.
- Move the database URI into .env and load it with dotenv.
- Confirm that local tests use the intended database and that credentials are not tracked.
- Commit and push the safe code changes.
- Create a Render Web Service connected to the GitHub repository.
- Configure the correct build/start commands and environment variables on Render.
- Wait for a successful deployment and copy the public URL.
- Test the public URL with Postman, then verify created data in Atlas.

10. Postman collections

The lecture also demonstrates saving API requests in a Postman collection. Collections help organize endpoints and reuse requests instead of rebuilding them each time.

- Create a collection named after the project, such as Hotel.
- Save requests with meaningful names, for example Get People, Get Managers, or Create Menu Item.
- Use the deployed URL for online testing and localhost for local testing.
- Do not save or share real passwords, API keys, or other secrets in exported collections.

A saved collection makes it easier to share the API workflow with teammates or test the same endpoints again later.

11. Key commands quick reference

Command	Purpose
git --version	Check whether Git is installed.
git init	Initialize Git in the current project.
git status	See tracked/untracked and staged/modified changes.
git add .	Stage eligible changes in the current directory.
git commit -m "message"	Create a local snapshot.
git remote -v	Show configured remotes.
git push	Send local commits to the remote repository.
git pull	Bring remote changes into the current branch.
npm install	Install dependencies from package metadata.
npm install dotenv	Add dotenv to load local environment variables.
npm start	Run the configured start script, if defined.

12. Revision questions

- What problem does Git solve, and what is a commit?
- What is the difference between git add, git commit, and git push?
- Why should node_modules generally not be pushed to GitHub?

- What does .gitignore do, and why does it not untrack files already committed?
- What is the difference between a local Git repository and a remote repository?
- What does git pull do when a collaborator has pushed changes?
- Why use MongoDB Atlas instead of relying only on a local database?
- Why should the database connection string be stored in an environment variable?
- What does dotenv do, and how does process.env access a variable?
- Why should the server use process.env.PORT || 3000?
- What should you check if a Render deployment fails?
- How can you verify that a deployed POST request wrote data to the cloud database?

13. Final takeaways

- Git tracks project history; GitHub stores the repository online.
- Commit working changes with meaningful messages, and push commits to keep the remote up to date.
- Use .gitignore to exclude node_modules, .env, and other local-only files.
- MongoDB Atlas hosts the database online; changing the URI changes which database the app targets.
- Use dotenv and process.env for configuration, and never publish real credentials.
- Configure the hosting platform’s environment variables and ensure the server uses the assigned port.
- Render deploys the API from the repository and gives it a public URL; test the deployed endpoints with Postman.

Scope note: These notes follow the supplied transcript. Hosting-provider dashboards, free-tier limits, and exact commands can change. Use the current platform UI and your project’s actual package.json, route names, and environment-variable names when applying the workflow.